Name: Dimple Acharya

Roll no. 23EJDAI026

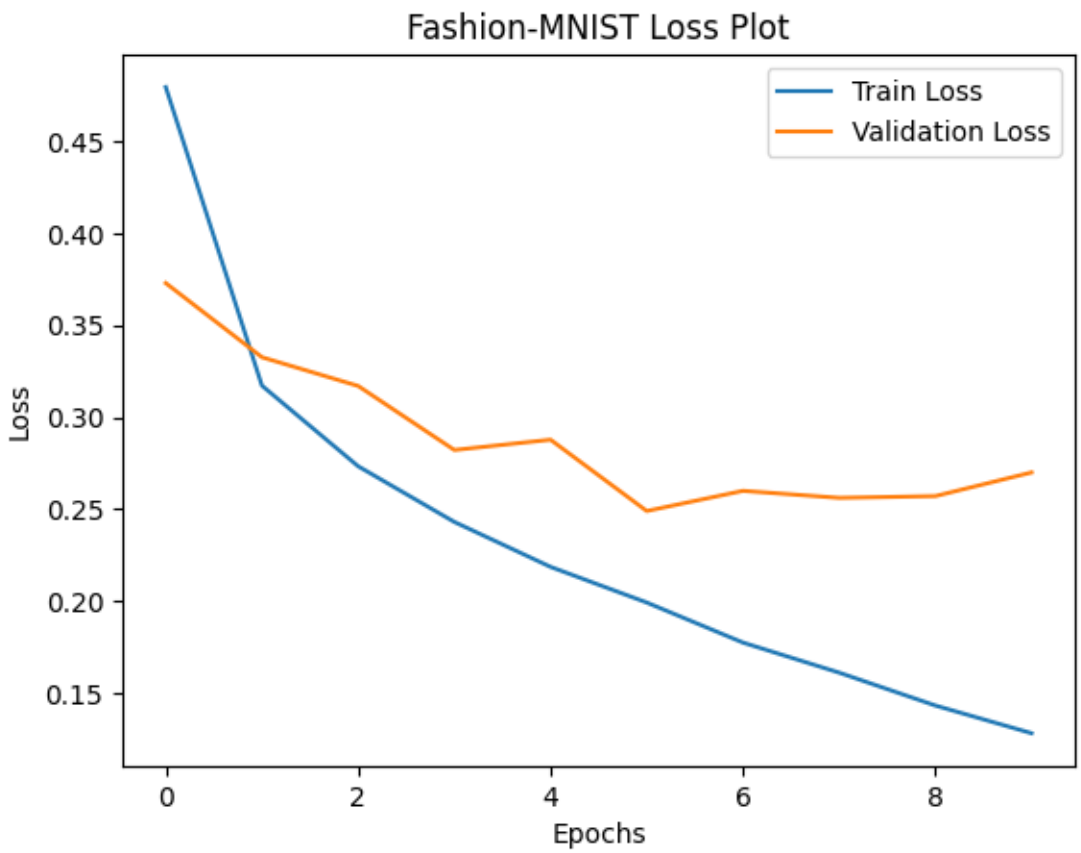
Date: 05-feb-2026

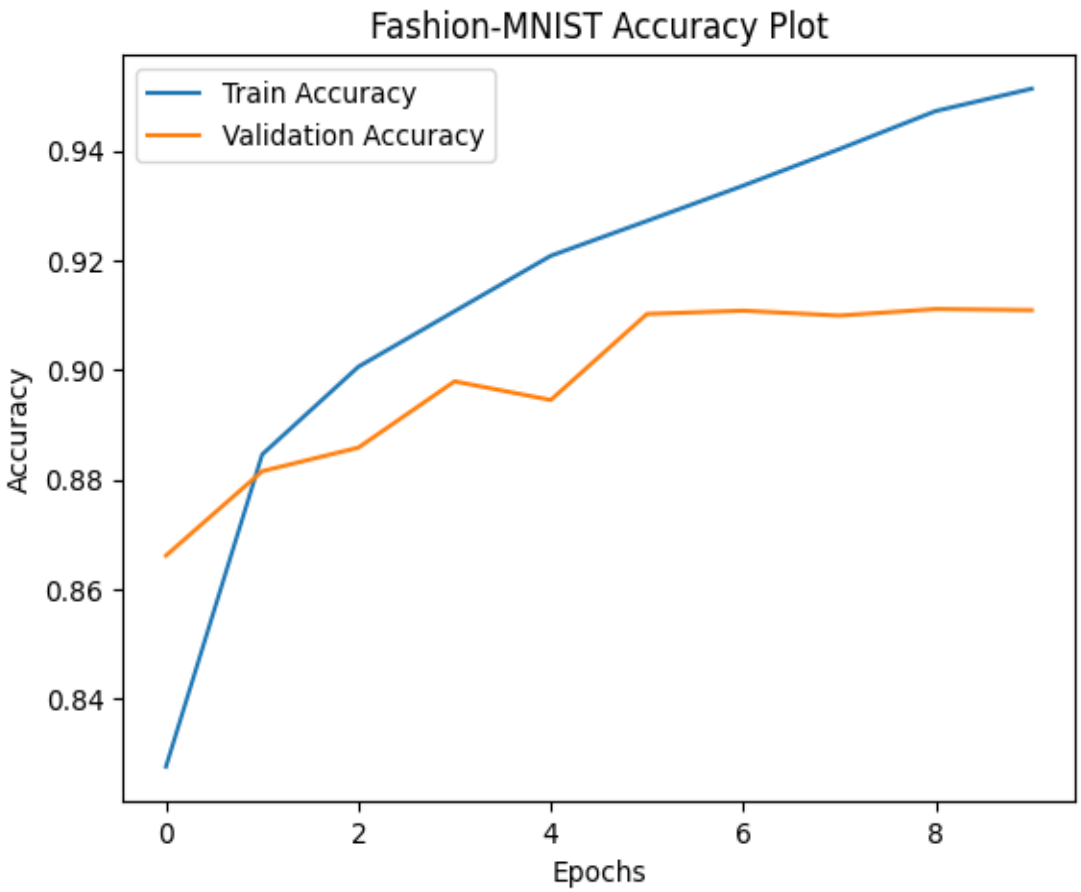
1)CNN for FASHION-MNIST CLASSIFICATION

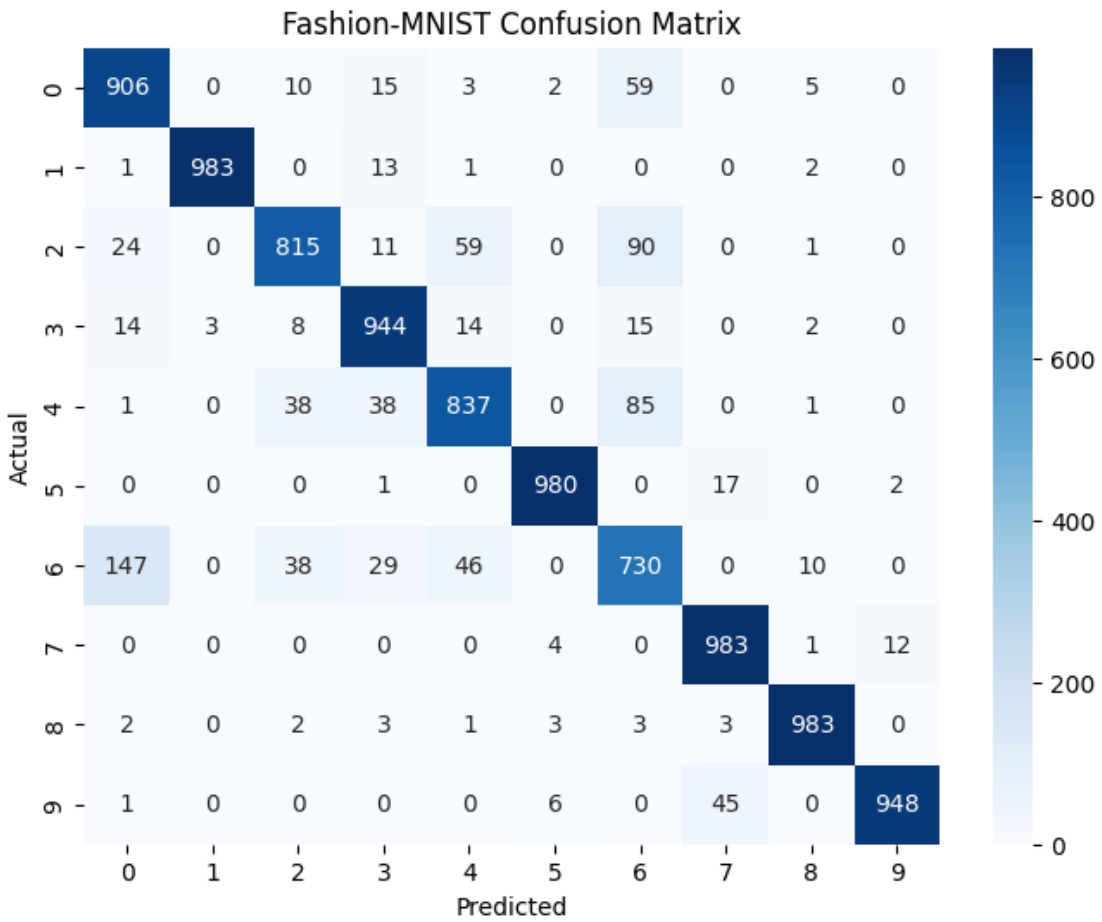
```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, classification_report

import seaborn as sns


# Load dataset

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()


# Normalize and reshape for CNN

x_train = x_train.reshape(-1, 28, 28, 1) / 255.0

x_test = x_test.reshape(-1, 28, 28, 1) / 255.0


# CNN Model

model = tf.keras.models.Sequential([

   tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),

   tf.keras.layers.MaxPooling2D((2,2)),


   tf.keras.layers.Conv2D(64, (3,3), activation='relu'),

   tf.keras.layers.MaxPooling2D((2,2)),


   tf.keras.layers.Flatten(),

   tf.keras.layers.Dense(128, activation='relu'),

   tf.keras.layers.Dense(10, activation='softmax')

])


# Compile
```

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
history = model.fit(x_train, y_train,
                    validation_data=(x_test, y_test),
                    epochs=10, batch_size=64)

# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
print("\nTest Accuracy:", test_acc)

# Accuracy Plot
plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Fashion-MNIST Accuracy Plot")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Loss Plot
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Fashion-MNIST Loss Plot")
```

```
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Confusion Matrix
y_pred = np.argmax(model.predict(x_test), axis=1)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Fashion-MNIST Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```


Fashion-MNIST Loss Plot
Train Loss
Validation Loss
Loss
0.45
0.40
0.35
0.30
0.25
0.20
0.15
0
2
4
6
8
Epochs


Fashion-MNIST Accuracy Plot
Train Accuracy
Validation Accuracy
Accuracy
0.94
0.92
0.90
0.88
0.86
0.84
0
2
4
6
8
Epochs



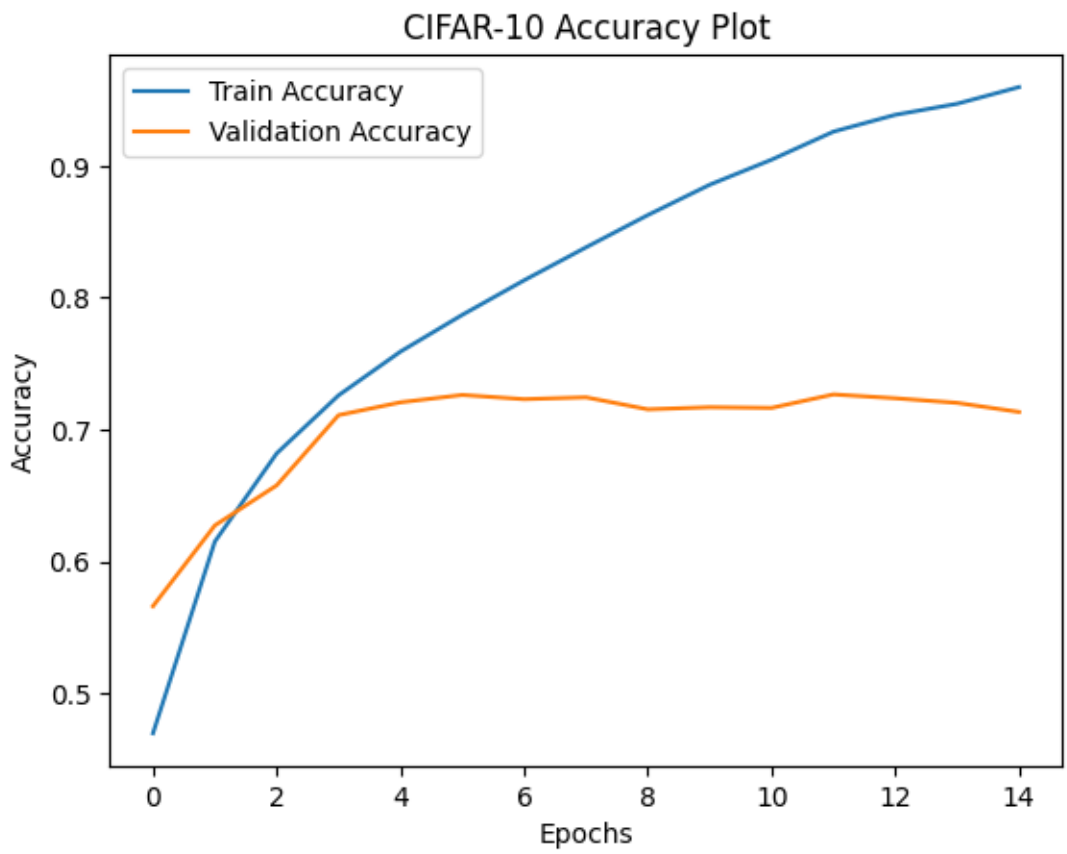

```
Classification Report:

              precision    recall  f1-score   support

           0       0.83      0.91      0.86      1000
           1       1.00      0.98      0.99      1000
           2       0.89      0.81      0.85      1000
           3       0.90      0.94      0.92      1000
           4       0.87      0.84      0.85      1000
           5       0.98      0.98      0.98      1000
           6       0.74      0.73      0.74      1000
           7       0.94      0.98      0.96      1000
           8       0.98      0.98      0.98      1000
           9       0.99      0.95      0.97      1000

    accuracy                           0.91     10000
   macro avg       0.91      0.91      0.91     10000
weighted avg       0.91      0.91      0.91     10000
```

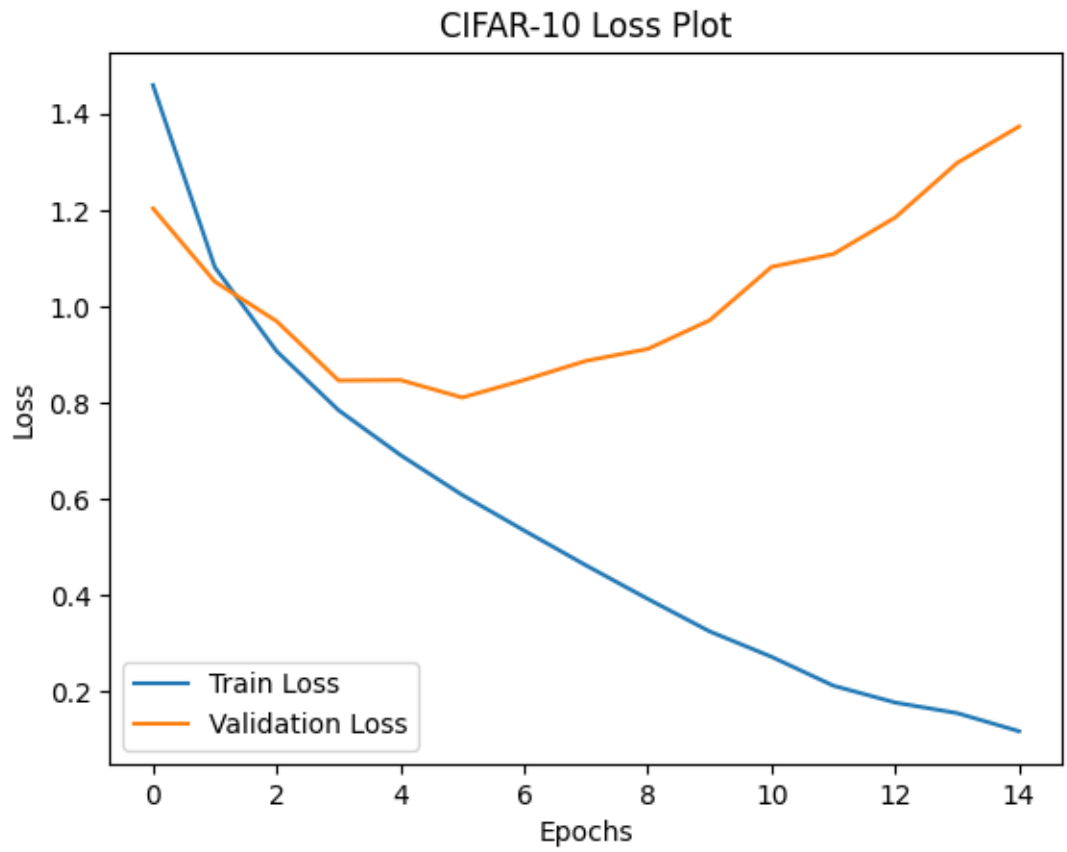
2)CNN for CIFAR-10 CLASSIFICATION

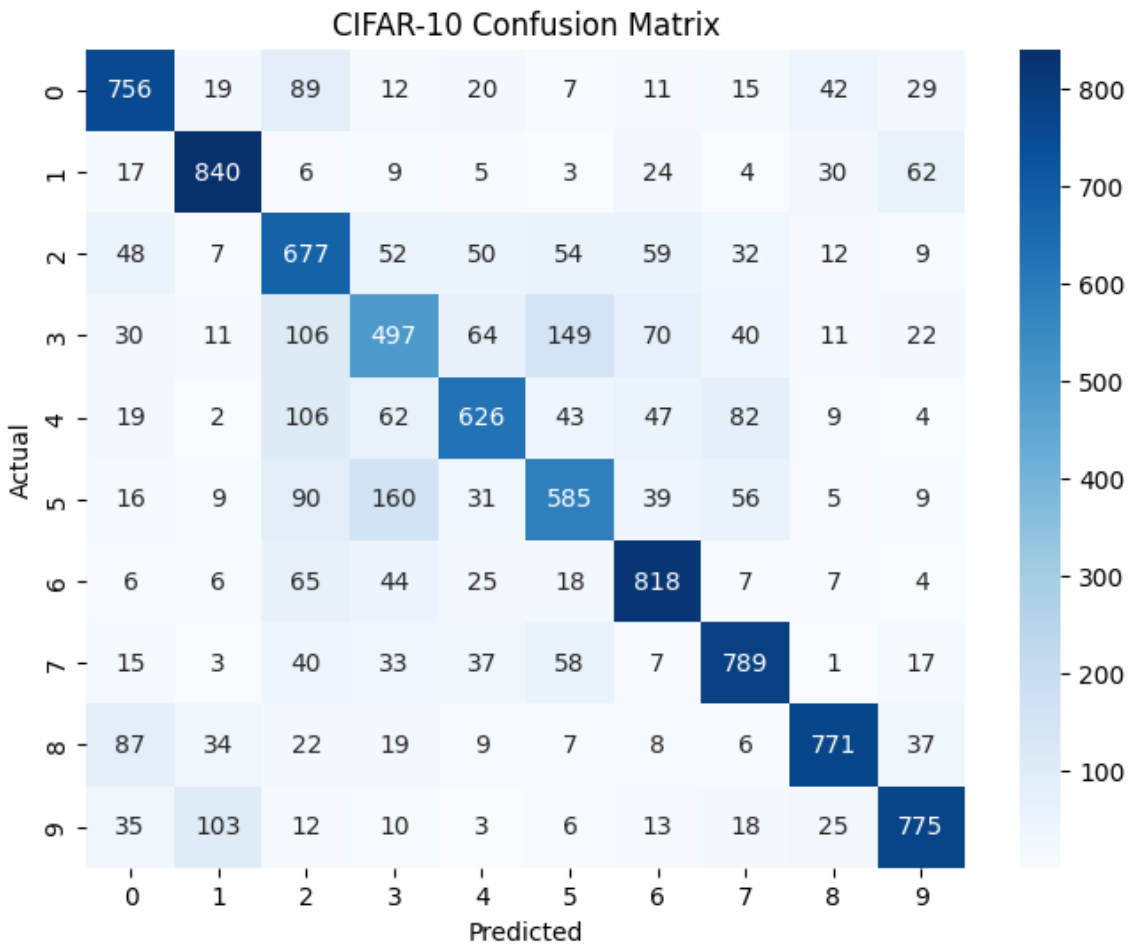
```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, classification_report

import seaborn as sns


# Load CIFAR-10

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()


# Normalize

x_train = x_train / 255.0

x_test = x_test / 255.0


# Convert labels to 1D

y_train = y_train.flatten()

y_test = y_test.flatten()


# CNN Model

model = tf.keras.models.Sequential([

   tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),

   tf.keras.layers.MaxPooling2D((2,2)),


   tf.keras.layers.Conv2D(64, (3,3), activation='relu'),

   tf.keras.layers.MaxPooling2D((2,2)),


   tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
```

```
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

# Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
history = model.fit(x_train, y_train,
                    validation_data=(x_test, y_test),
                    epochs=15, batch_size=64)

# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
print("\nTest Accuracy:", test_acc)

# Accuracy Plot
plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("CIFAR-10 Accuracy Plot")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```
#Loss Plot
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("CIFAR-10 Loss Plot")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Confusion Matrix
y_pred = np.argmax(model.predict(x_test), axis=1)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("CIFAR-10 Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```


CIFAR-10 Accuracy Plot
Train Accuracy
Validation Accuracy
Accuracy
0.5
0.6
0.7
0.8
0.9
0
2
4
6
8
10
12
14
Epochs


CIFAR-10 Loss Plot
Train Loss
Validation Loss
Loss
0.2
0.4
0.6
0.8
1.0
1.2
1.4
0
2
4
6
8
10
12
14
Epochs




```
Classification Report:

              precision    recall  f1-score   support

           0       0.73      0.76      0.75      1000
           1       0.81      0.84      0.83      1000
           2       0.56      0.68      0.61      1000
           3       0.55      0.50      0.52      1000
           4       0.72      0.63      0.67      1000
           5       0.63      0.58      0.61      1000
           6       0.75      0.82      0.78      1000
           7       0.75      0.79      0.77      1000
           8       0.84      0.77      0.81      1000
           9       0.80      0.78      0.79      1000

    accuracy                           0.71     10000
   macro avg       0.72      0.71      0.71     10000
weighted avg       0.72      0.71      0.71     10000
```

Google Colab Link

https://colab.research.google.com/drive/1VwE5Sd874romN-sQksEt-07DnGw_4lt7?usp=sharing